

Week 3 Capstone Progress Update

This Week's Focus

My responsibility is the dashboard. This week I focused on getting it ready to receive the trained detection model from Cole, so that when his work is done, integration is fast and clean.

The dashboard currently runs on simulated data. The goal was not to add more visual features, but to restructure things so the simulated data can be swapped out with real model output without breaking the rest of the app.

Starting Point

Before this week, the dashboard already had the right structure for what we want to display: attack type, timestamp, source and destination, severity, and confidence score. The filters and visualizations were working. The issue was that the underlying data flow was hardcoded, which would have made the model handoff messy.

Changes Made

Isolated the data source. The simulated data was previously generated inline throughout the app. I moved it into a single function so that swapping in the real model output only requires changing one place.

Restructured the model info panel. The model name, dataset, and accuracy were hardcoded as placeholder values inside the layout. Moved them into a config object that we can update once Cole finalizes his metrics. Anything not yet known displays as "Pending."

Added caching on data loads. Without it, the app would re-process data on every interaction. With it, results refresh on a 60-second interval, which matches what we'd want for live monitoring.

Made the filters dynamic. The attack type and severity filters used to read from a fixed list. They now derive their options from whatever data is loaded, so any new attack class Cole's model produces will appear automatically without code changes.

Open Questions for Cole

Three things I need from him before final integration:

- The format of the handoff: trained model file, batch predictions, or live API.
- The exact attack class labels his model outputs, so they match my severity and color mappings.
- Whether the dashboard should run inference itself or just display results he generates.

Status

Dashboard is structurally ready. Once Cole hands off the model and we align on the three items above, integration should be quick since the rest of the app is already designed to pull from a single, swappable source.